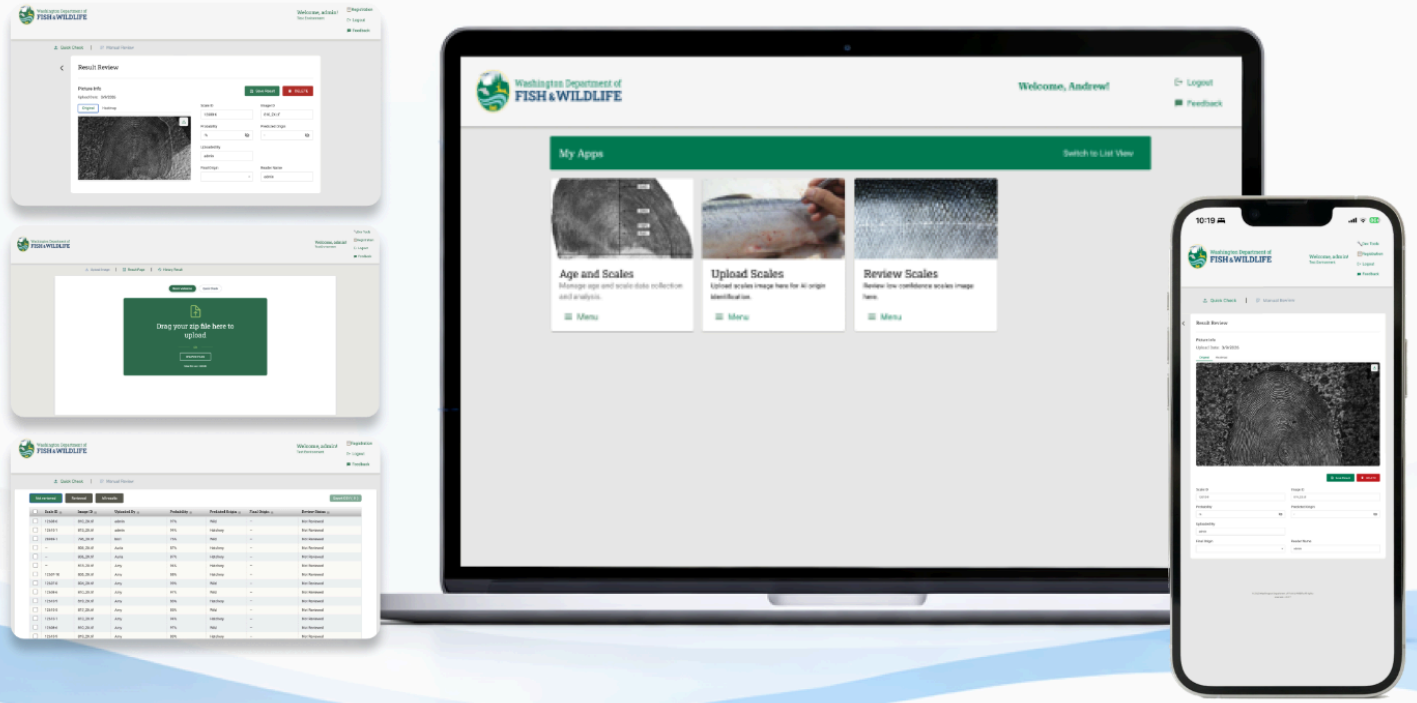
Salmon Says

Fish Origin identification System

Version 1.0



Team Members

Auria Zhang,
Diyun Lu,
Yunqing Zhao

Industrial Sponsor

Washington Department of Fish and Wildlife & AWS

Instructor

Luyao Niu,
Mark Licata

Submission Date

March 12, 2026

MASTER OF SCIENCE IN TECHNOLOGY INNOVATION
UNIVERSITY of WASHINGTON

aws

Washington Department of FISH & WILDLIFE

Product Overview

Salmon Says is a web-based system designed to support fish origin identification using fish scale images. The platform helps determine whether a salmon is hatchery-origin or wild by combining automated prediction with expert validation. By bringing image upload, prediction results, and review tools into one system, Salmon Says provides a more organized and efficient workflow for fish scale analysis.

Traditionally, identifying fish origin requires trained specialists to manually examine scale patterns under a microscope. This process can be time-consuming and difficult to scale when large numbers of samples are collected during field surveys. Salmon Says helps streamline this process by providing an initial automated prediction and a centralized platform to manage images and results.

The system is designed for two main user groups, field technicians, who pick up fish scales from live fish and make them as fish scale cards, upload fish scale images and initiate prediction, biologists, who review results and perform manual validation when necessary. The platform supports image uploads, automated origin predictions, visual explanations of model results, and a structured review workflow. Historical records are also stored so users can track previous uploads and validation outcomes.

By integrating prediction, visualization, and expert review into a single interface, Salmon Says helps reduce manual workload while maintaining a reliable and transparent identification process.

System Requirements

The Fish Origin Identification System supports local demonstration, local development, and server-backed operation. The application can run in a web browser for interface demonstration, with a local Node.js server for authentication and user management, or with a full local Python backend for inference and data storage integration. The **production version** of the Fish Origin Identification System is deployed on AWS. The frontend is a static web application hosted in Amazon S3 and delivered through Amazon CloudFront. The backend API is hosted on an Amazon EC2 instance running Ubuntu. End users access the system through a web browser and do not need to install the application locally.

Operating Systems

- For deployed backend hosting: Ubuntu Linux on Amazon EC2
- For end users accessing the deployed system: Windows, macOS, or Linux
- For local development and testing: Windows, macOS, or Linux

Software Dependencies

- A modern web browser such as Google Chrome, Microsoft Edge, Firefox, or Safari
- For local web server setup:
    - Install Node.js and npm, then verify the installation with: node -v && npm -v
    - You can go to https://nodejs.org for Windows and MacOS version
    - Production version of node and npm is v24.14.0 and 11.9.0
- For local backend inference setup: Using a virtual environment with Python 3.12 is recommended for local setup. All required packages can be installed from the provided requirements file using: pip install -r /home/ubuntu/Fish-App/Fish-App-main/salmon-backend/requirements.freeze.txt.

- For **deployed hosting** and maintenance:
    - Ubuntu server environment on EC2
    - AWS services such as S3, CloudFront, DynamoDB, and EC2

Internet Requirements

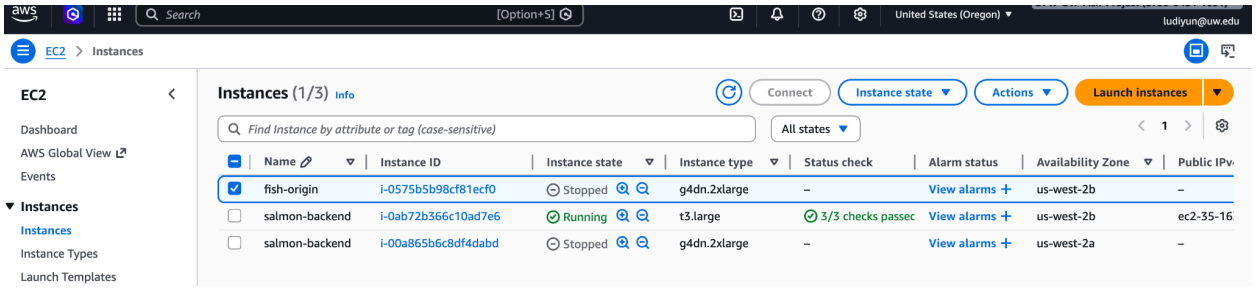
- For the **deployed production system**, internet access is required because the frontend and backend are hosted on AWS and accessed through a web browser
- For local development or local testing, internet access is not strictly required if all files, dependencies, and backend services are already installed and configured locally
- Internet access may still be required during setup to install packages, download dependencies, or access AWS-hosted resources

Installation & Setup Instructions

All codes are managed through GitHub, in a private repository: Fish-App

For Deployed Production Version

- Navigate to EC2 instances in the Oregon region, check the server "salmon-backend" that is type t3.large is running. Click on the Instance ID and go to the detail page. Click on connect to connect through the AWS webpage or using SSH.



- After connecting to the EC2, check whether the Flask backend service is running. Navigate to the ~/Fish-App directory in the terminal and use command: sudo systemctl status salmon-backend
  If the status is active (running), then no other actions are required.

Otherwise, you can restart the service by: sudo systemctl restart salmon-backend

  Then you can check the status again

- Then, in the same directory, check if the login server is running: sudo systemctl status fish-node
  If needs to restart: sudo systemctl restart fish-node
- After that, you can access the system using the domain: https://salmonsays.site/
- If you want to test the application locally on the server instead of using the public domain, open a browser on the server or create an SSH tunnel, then access: http://localhost:4000

  If using SSH port forwarding from a local machine, run:

  ssh -L 4000:localhost:4000 ubuntu@ec2-35-163-47-188.us-west-2.compute.amazonaws.com

  Then open the following URL in your local browser:

  http://localhost:4000

This allows the deployed Node.js web server to be tested locally without going through the public domain.

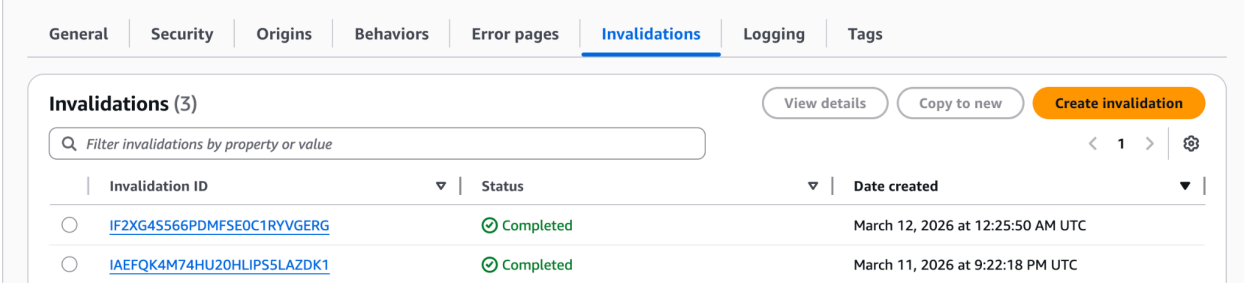
For Local Version

If local testing is needed on your own computer, first make sure the required frontend and backend services are installed and configured. Then start the local server processes and open the application in a browser.

For Setting Up New Codes

The frontend static files are hosted in an Amazon S3 bucket and delivered through Amazon CloudFront, which manages the public domain. The backend services are hosted on the EC2 instance named salmon-backend. The deployment pipeline is not yet automated. Therefore, whenever the frontend code is updated, the new static files must be uploaded manually to the S3 bucket, and the CloudFront distribution cache must be invalidated so that the latest version is served to users. You can use the command in the terminal to upload the corresponding code to S3:
aws s3 cp /path/to/frontend s3://<bucket-name>/ --recursive

For CloudFront, you can go to the AWS console and navigate to the only distribution and go to the invalidation tab to create new invalidations, which include the directory that needs to be refreshed.



For the backend codes, remember to restart the corresponding services for the new codes to take effect.

Operating Instructions

To start using the system, open a web browser and navigate to the deployed website:

https://salmonsays.site/

The login page should appear. Enter a valid username and password, then click Sign In. New users who do not yet have an account should use the account creation option and wait for administrator approval before logging in. Administrators can approve the requests from their portal.

After login, the user is taken to the main application interface. From the homepage, users can access the primary features of the system, including the Upload Scales portal and Review Scales portal. You can upload the scales in batch or using one image in the upload scales portal. After successfully uploading the images, the results will be displayed in the results tab, and you can access all uploaded history results in the History Results tab.

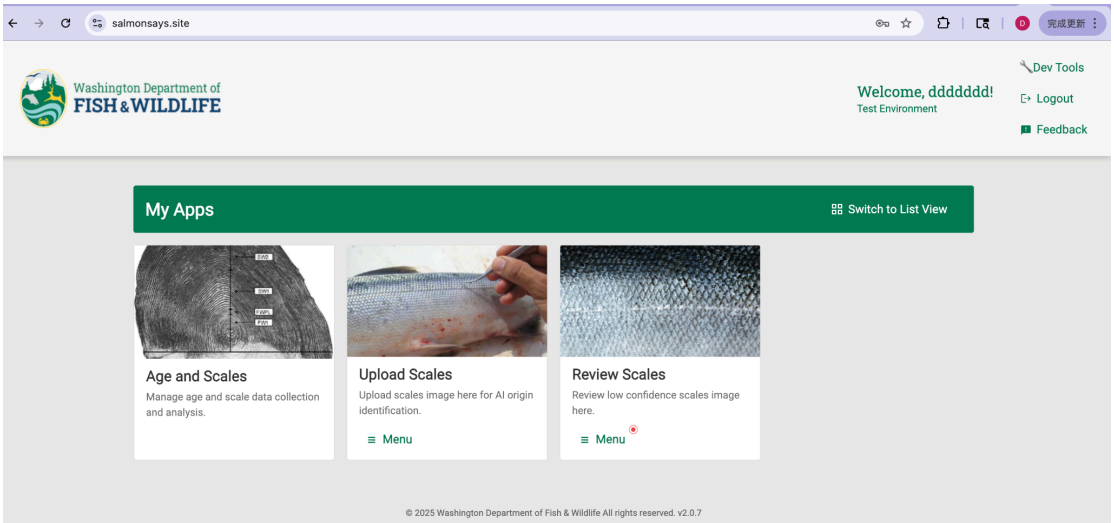


Image: Portal Choosing Page

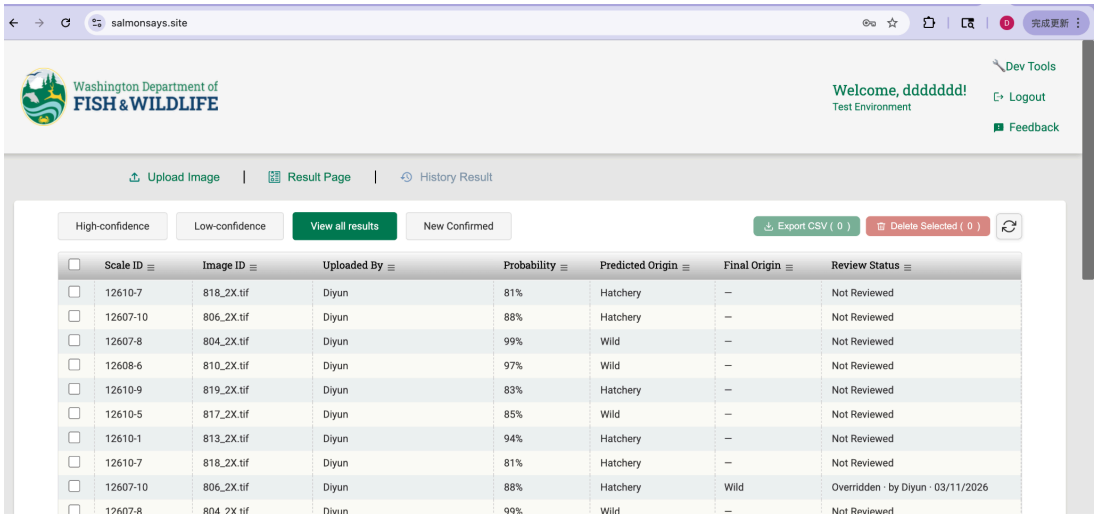


Image: History Result Page

Users can choose images of low confidence or the results they don't trust to commit for further review. After being committed, the results will be displayed in another portal's Not Reviewed tab. When experts entered detail page, modify and save the results, it will be in the Reviewed tab, and the results will be updated to the History Result tab.

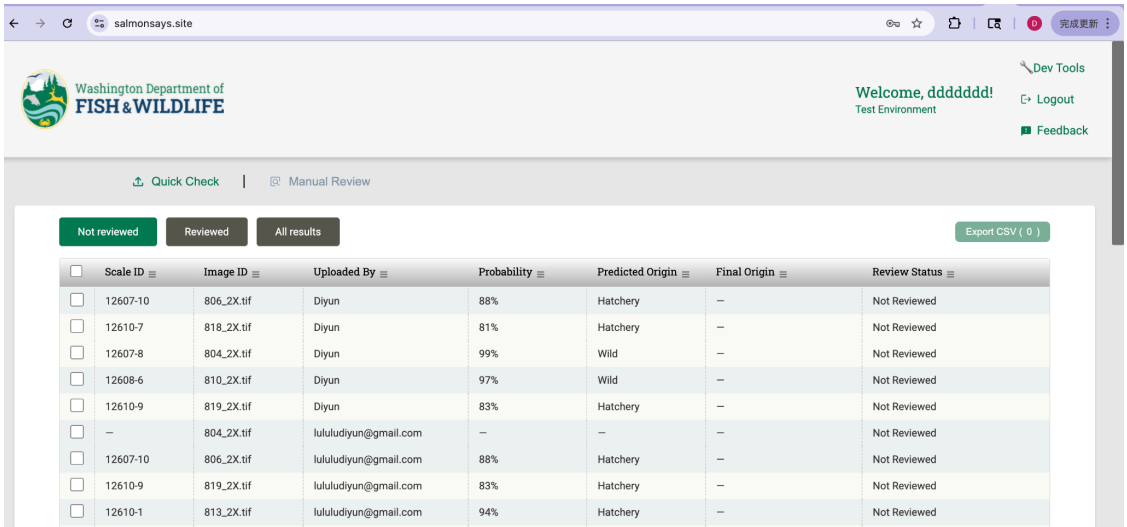


Image: Not Reviewed Tab

When you need to run through an image for a quick check, you can navigate to the Quick Check Tab, upload one image, and enter the fork length and sex manually to get a result quickly.

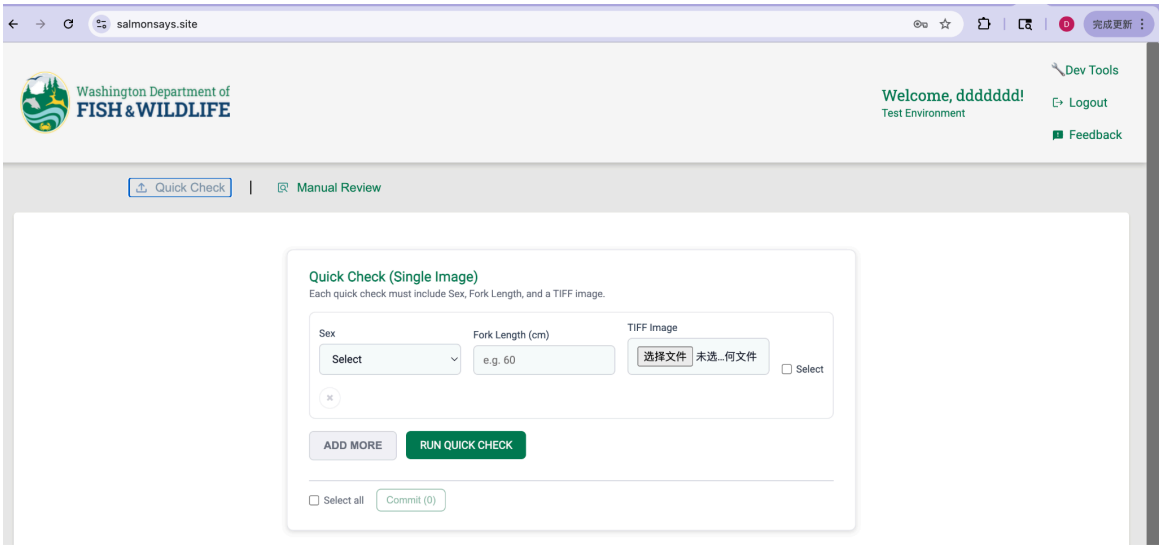


Image: Quick Check Tab

To properly shut down a user session, click the Logout button in the application interface. This ends the current session securely. After logging out, the browser window or tab may be closed.

For administrators or maintainers, if a local or server-hosted test session is being used, the system services may be stopped after use if required. For systemd-managed services, use:

```
sudo systemctl stop salmon-backend
sudo systemctl stop fish-node
```

These commands should only be used when intentionally shutting down the backend services for maintenance or testing, not during normal user operation.

System Architecture

Salmon Says is a web-based verification platform that supports fish origin identification through image uploads, automated predictions, and expert reviews. The system connects field data collection with laboratory validation in a unified workflow.

The system consists of several integrated components:

Frontend

The frontend is a browser-based interface built using HTML, CSS, and JavaScript. It is hosted on Amazon S3 using static website hosting, with the custom domain https://salmonsays.site/ configured for user access.

The interface allows users to upload fish scale images, perform quick predictions, view historical results, and access manual review tools.

Backend API

The backend is a Python Flask API deployed on an Amazon EC2 instance. The backend processes image upload requests, performs prediction inference, manages workflow logic, and communicates with cloud storage and database services.

The backend service runs as a managed system service and exposes REST endpoints that are called by the frontend application.

Prediction Processing

Machine learning inference is performed directly on the EC2 instance using PyTorch. The backend loads a trained model checkpoint from local storage and performs predictions within the same process.

For each uploaded fish scale image, the system generates:

- a predicted origin label
- a confidence score
- a Grad-CAM visualization overlay

The Grad-CAM heatmap highlights image regions that influenced the model's prediction and is saved as an overlay image.

Validation Service

Upload validation is handled through a serverless endpoint using AWS API Gateway and Lambda. The frontend calls the validation endpoint before image submission to verify upload requirements.

Cloud Storage

Fish scale images and generated visualization results are stored in Amazon S3. The backend uploads original images and generated overlays to structured S3 paths for persistent storage.

Technologies Used

The system is built using the following technologies:

Frontend

- HTML
- CSS
- JavaScript
- Amazon S3 static website hosting

Backend

- Python
- Flask REST API
- PyTorch for local model inference

Cloud Infrastructure

- Amazon EC2 for backend service hosting
- Amazon S3 for frontend hosting, uploaded images, and generated overlays
- AWS API Gateway and AWS Lambda for upload validation services
- Amazon DynamoDB for storing system metadata and workflow records

Version Control

- GitHub for source code management and collaboration

User Roles

The system currently supports three user roles:

- **Technician**: upload images, run Quick Check, and view history results
- **Biologist**: perform manual review and make final origin determinations
- **Admin**: approve or reject new user registration requests

The Technician and Biologist roles are available in the main system interface. Admin approval is handled through a separate administrative service.

Image Storage

Uploaded fish-scale images and generated visualizations are stored in Amazon S3.

The backend uploads the following files to the configured inference bucket:

- original TIFF images
- processed PNG display images
- Grad-CAM overlay visualizations

Example storage paths follow the structure:

inference-results/jobs/{job_id}/...

These files are referenced in the database through stored S3 keys.

Database Information

The system uses Amazon DynamoDB to store prediction metadata and workflow records.

The DynamoDB table stores one record per image, using the following primary keys:

- **Partition Key:** job_id
- **Sort Key:** image_id

Each record contains metadata describing the prediction results and review workflow.

Stored attributes include:

- job_id – job identifier
- image_id – image identifier
- scale_id – scale sample identifier
- user_id – user who uploaded the image
- created_at – upload timestamp
- updated_at – last update timestamp
- raw_key – S3 path to the original image
- overlay_key – S3 path to the Grad-CAM overlay image

- pred_label – predicted origin (0 = Hatchery, 1 = Wild)
- confidence – prediction probability
- fork_length – recorded fish fork length
- sex – recorded fish sex
- review_status – current review state
- submitted_to_lab – whether the record was sent for manual review
- manual_read_origin – final expert determination
- reader_name – reviewer name
- reviewer_id – reviewer identifier

Additional workflow fields may include:

- field_acknowledged_at
- field_acknowledged_by

These fields track when field users have acknowledged finalized results.

The DynamoDB table stores metadata only. All image files are stored in S3.

Data Flow

The primary system workflow follows this sequence:

1. A technician uploads fish scale images through the web interface.
2. The frontend sends the image data to the Flask backend API, with image content encoded in the JSON request body.
3. The backend performs model inference using PyTorch.
4. A Grad-CAM overlay visualization is generated.
5. Images and overlays are uploaded to Amazon S3.
6. Metadata and prediction results are stored in DynamoDB.
7. Results become available in the History and Manual Review interfaces.

Troubleshooting

The following issues may occur during system operation.

Image upload failure

Possible causes include network interruption, unsupported file format, invalid upload structure, or exceeding upload size limits.

Recommended actions:

- Ensure images are in supported formats such as TIFF or ZIP, for batch uploads.
- Verify that the upload follows the required structure and metadata requirements.
- Check that the request size does not exceed system limits.

Backend service unreachable

If the frontend cannot connect to the prediction service, the backend EC2 instance may be unavailable.

Recommended actions:

- Verify that the EC2 instance is running.
- Confirm that the backend service is active.
- Check server logs for runtime or dependency errors.

CORS or network access error

If the browser reports a CORS error or the frontend cannot reach the backend API, verify that the backend CORS configuration includes the frontend domain and that the EC2 security group allows inbound traffic on the required port.

Upload validation failure

Upload validation is performed through an API Gateway endpoint backed by AWS Lambda. If validation fails, the request may be rejected before submission.

Recommended actions:

- Ensure the request format matches validation requirements.

- Check that the validation endpoint is available and reachable.

Prediction processing delay

Large batch uploads may require significant processing time because each image must go through model inference, Grad-CAM generation, S3 upload, and DynamoDB write operations.

Recommended actions:

- Upload smaller batches when possible.
- Allow sufficient time for processing before refreshing results.

Database write errors

If metadata cannot be written to DynamoDB, prediction results may not appear in the history table.

Recommended actions:

- Check backend logs for DynamoDB errors.
- Verify that backend IAM permissions are configured correctly.

Authentication or session expiration

Users may be logged out if their authentication token expires.

Recommended actions:

- Re-authenticate and log in again.
- Confirm that the account has already been approved if the login fails after registration.

Limitations

The current system includes several known limitations.

Limited training dataset

The prediction model was trained using a limited dataset of fish scale images. Certain patterns may still produce uncertain predictions and require expert review.

Confidence threshold tuning

The system uses a confidence threshold to distinguish between higher-confidence predictions and cases requiring manual validation. This threshold is still under evaluation and may change as more data becomes available.

Processing latency

Inference and visualization generation are performed on the EC2 instance using PyTorch. Large image batches can introduce noticeable processing delays.

Duplicate detection limitations

The current duplicate checking logic primarily compares image identifiers and historical records. Duplicate files submitted in different contexts may not always be detected.

Upload size constraints

Upload requests are constrained by backend and infrastructure limits, including request size limits and batch processing capacity.

Access control architecture

User role management currently exists in a separate administrative service. Full integration between the authentication system and backend workflow services is still under development.

Role separation is not fully enforced in one unified backend

Although Technician, Biologist, and Admin responsibilities are defined, the current architecture does not yet provide a fully unified access-control implementation across all services.